

WORLD 2 - 1

PROJECT 1

HEALTH INSURANCE CHATBOT

HEALTHCARE / INSURANCE / FINTECH

BUILD TIME
~90 min

DIFFICULTY
MEDIUM

KEY CONCEPT
RAG +
MULTI-TOOL

THE ELEVATOR PITCH

Build a conversational AI agent that lets a customer upload their health insurance policy PDF and ask plain-English questions about coverage, claim amounts, network hospitals, and required documents. The agent uses RAG over the policy document plus 5-6 calculator and lookup tools to give complete, accurate answers - the kind of system every major Indian insurer is shipping in 2026.



THE BUSINESS PROBLEM

Health insurance policies are notoriously hard to read. A typical policy document runs 30-50 pages of dense legal language with sub-limits, waiting periods, co-pays, exclusions, and network rules. Customers regularly call support to ask the same questions: 'Is XYZ covered?', 'How much will I get for this surgery?', 'Which hospital can I go to?'. Each call costs an insurer between Rs. 200 and Rs. 500 in agent time, and these calls represent 60-70% of total support volume at most insurers. Deflecting them with an AI bot is a direct line to the bottom line.

WHY NOW

Market context: India's health insurance market crossed Rs. 1.1 lakh crore in premiums in FY25. AI-driven customer service is one of the top 3 spend areas for insurers in 2026 - every major insurer has either built or is buying conversational AI capabilities. Companies like Acko, Plum, and PolicyBazaar have AI assistants in production right now.

Companies actively building this:

Insurers: HDFC Ergo, Star Health, Niva Bupa, ICICI Lombard, Acko, Tata AIG, Bajaj Allianz, ManipalCigna, Care Health. **Insurtech:** Plum, PolicyBazaar, Coverfox, Toffee, Onsurity, Vital. **Adjacent:** Razorpay (insurance), Cred (insurance offerings), Paytm Insurance.

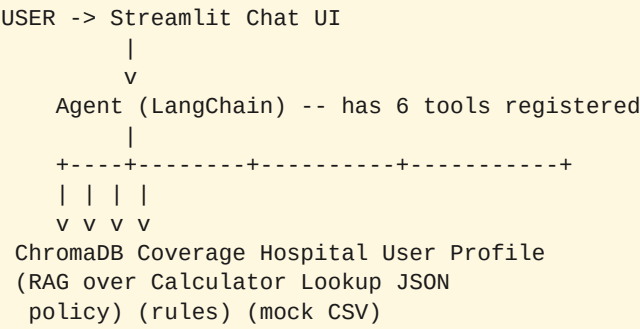
WHAT YOU'LL BUILD

A Streamlit chat interface where users upload their insurance policy PDF, ask natural-language questions, and receive complete, accurate answers. The agent decides which combination of RAG search and lookup tools to use based on the question. The system is multi-turn - users can refine, ask follow-ups, and drill into details - and shows its reasoning so users trust the answers.

User stories (what your agent must support):

- "As a policy holder, I want to know if cataract surgery is covered, and how much I'll get back."
- "As a policy holder, I want to find a cashless network hospital near me for a specific specialty."
- "As a policy holder, I want to know the documents I need to file a claim for hospitalization."
- "As a policy holder, I want to compare my pre-existing condition coverage to the standard policy."
- "As a policy holder, I want to track the status of an open claim by claim ID."

ARCHITECTURE OVERVIEW



AGENT'S TOOLBOX

The agent picks one or more of these tools per question. Each tool has a clear name, a description (the LLM reads this to decide when to call it), and typed inputs.

TOOL	WHAT IT DOES	INPUTS
search_policy_docs	RAG search over the uploaded policy PDF - returns top relevant chunks	query: str
check_coverage	Returns whether a specific condition or procedure is covered	condition: str
calculate_claim	Computes expected claim amount given the bill and policy rules	condition: str, bill_amount: float
get_user_policy	Returns user's policy details: sum insured, co-pay, no-claim bonus	user_id: str
find_network_hospitals	Lists cashless network hospitals in a city for a specialty	city: str, specialty: str
list_required_documents	Returns paperwork checklist for a specific claim type	claim_type: str

Example calls and returns

search_policy_docs

Call: search_policy_docs(query='waiting period for pre-existing diseases')

Returns: [{chunk: 'Pre-existing diseases have a 3-year waiting period from policy start date.', page: 12}]

check_coverage

Call: check_coverage(condition='cataract surgery')

Returns: {covered: True, sub_limit: 40000, co_pay: 0.10, waiting_period: 0}

calculate_claim

Call: calculate_claim(condition='cataract', bill_amount=50000)

Returns: {covered_amount: 40000, co_pay: 4000, payable_to_user: 36000}

SAMPLE CONVERSATIONS

Below are 2-3 different conversational paths your agent should handle. Notice how the agent decides which tool to call based on the user's intent.

Flow: Coverage check + claim calculation

COVERAGE CHECK + CLAIM CALCULATION	
USER	I'm getting cataract surgery at Apollo Bangalore next month. How much will I get?
TOOL	check_coverage('cataract surgery') -> covered, sub-limit Rs. 40,000
TOOL	find_network_hospitals('Bangalore', 'ophthalmology') -> Apollo is cashless
TOOL	get_user_policy(user_id) -> 10% co-pay
TOOL	calculate_claim('cataract', 50000) -> Rs. 36,000 payable
AGENT	Cataract surgery is covered up to Rs. 40,000 under your policy. Apollo Bangalore is in your cashless network. With your 10% co-pay, you'll get approximately Rs. 36,000 reimbursed (assuming the bill is ~Rs. 50,000).

Flow: Document checklist for a claim

DOCUMENT CHECKLIST FOR A CLAIM	
USER	What documents do I need to file a hospitalization claim?
TOOL	list_required_documents('hospitalization')
AGENT	You'll need: (1) Pre-authorization form from the hospital, (2) Original discharge summary, (3) All bills + receipts, (4) Diagnostic reports (X-rays, blood work), (5) ID proof. For cashless claims, the hospital handles most of this for you. For reimbursement claims, you submit everything within 30 days of discharge.
USER	What if I forget to submit within 30 days?
TOOL	search_policy_docs('late claim submission')
AGENT	Your policy allows late submissions up to 60 days with a written explanation. Beyond 60 days, claims are typically denied unless there's a medical emergency justification.

Flow: Comparing pre-existing condition coverage

COMPARING PRE-EXISTING CONDITION COVERAGE	
USER	I have diabetes. Is it covered?
TOOL	<code>search_policy_docs('diabetes pre-existing condition waiting period')</code>
TOOL	<code>check_coverage('diabetes')</code>
AGENT	Diabetes is classified as a pre-existing condition. Your policy has a 3-year waiting period for pre-existing conditions starting from policy start date. After that, related complications (diabetic retinopathy, kidney issues, etc.) are fully covered.

BUILD GUIDE

Recommended folder structure

PROJECT FOLDER

```
health_insurance_bot/  
  
+-- app.py # Streamlit chat UI entry point  
+-- agent.py # Agent setup, tool registration  
+-- tools/  
| +-- policy_search.py # RAG tool over the PDF  
| +-- coverage.py # check_coverage + calculate_claim  
| +-- hospitals.py # find_network_hospitals  
| +-- user.py # get_user_policy  
| +-- documents.py # list_required_documents  
+-- data/  
| +-- policy.pdf # Sample policy PDF  
| +-- hospitals.csv # Mock hospital network  
| +-- users.json # Mock user profiles  
| +-- coverage_rules.json # Coverage + sub-limits config  
+-- requirements.txt  
+-- README.md
```

Step-by-step

1. Sign up for free LLM API: **console.groq.com** (free, no card). Save the API key.
2. Install dependencies: `pip install langchain langchain-groq chromadb pdfplumber streamlit pandas`
3. Create the policy ingestion: parse PDF with pdfplumber, chunk into 500-token pieces, store in ChromaDB with embeddings.
4. Build mock data: `hospitals.csv` (50 entries) + `users.json` (3 sample profiles) + `coverage_rules.json` (20 conditions with sub-limits).
5. Implement the 6 tools as plain Python functions with type hints and clear docstrings.
6. Register tools with LangChain Agent. Use `create_tool_calling_agent` with Groq's `llama-3.3-70b` or similar.
7. Build the Streamlit chat UI - use `st.chat_message` + `st.chat_input`. Maintain history in `st.session_state`.
8. Test the 3 sample flows above. Print agent reasoning with `verbose=True`.

9. Polish: add file upload widget for the policy PDF, add a 'show reasoning' toggle.
10. Push to GitHub with a clean README. Include 1-2 screenshots and a sample policy.

Skeleton code

agent.py

```
from langchain.agents import create_tool_calling_agent, AgentExecutor

from langchain_groq import ChatGroq

from langchain.tools import tool

import streamlit as st, json


@tool

def check_coverage(condition: str) -> dict:

    rules = json.load(open('data/coverage_rules.json'))

    return rules.get(condition.lower(), {'covered': False})


# ... define the other 5 tools similarly ...


llm = ChatGroq(model='llama-3.3-70b-versatile')

tools = [search_policy_docs, check_coverage, calculate_claim,

         get_user_policy, find_network_hospitals, list_required_documents]


agent = create_tool_calling_agent(llm, tools, prompt)

executor = AgentExecutor(agent=agent, tools=tools,

                        verbose=True, max_iterations=8)


# Streamlit chat

if user_input := st.chat_input():

    response = executor.invoke({'input': user_input})

    st.chat_message('agent').write(response['output'])
```


RESOURCES & DATA

- **Sample policy PDFs:** Search 'health insurance policy wording site:hdfcergo.com filetype:pdf' (any major insurer publishes these for free)
- **Hospital data:** Use a CSV with 30-50 hospitals - name, city, area, specialties, cashless flag, rating
- **Mock users:** JSON with 3 fake user profiles (different sum insured, co-pay, NCB combinations)
- **Coverage rules:** JSON dict of 20-30 conditions to {covered, sub_limit, co_pay, waiting_period}
- **Groq API:** <https://console.groq.com> - free, very fast, no payment needed
- **LangChain docs:** <https://python.langchain.com> - tool calling examples

STRETCH GOALS

Finished early? Add one of these power-ups:

- + Add a **claim status tracker** tool that takes a claim ID and returns mock progress (filed -> processing -> approved/rejected)
- + Add a **compare policies** feature - upload 2 PDFs and ask 'which one covers X better?'
- + Voice input via the browser's Web Speech API (free, built-in)
- + Generate a downloadable **claim summary PDF** the user can save
- + Add **multi-language support** (Hindi, Tamil, Telugu) - prompt the LLM in the user's language
- + Add **medical jargon explanation** - hover over any medical term to get a plain-English definition

WATCH OUT FOR

Common pitfalls specific to this project:

- **Chunk size too large** — Keep PDF chunks to ~500 tokens. Larger chunks dilute relevance scoring and waste context.
- **Tool descriptions are vague** — Write tool descriptions as if explaining to a junior developer. The LLM picks the wrong tool when descriptions overlap.
- **No max_iterations** — Set max_iterations=8 in AgentExecutor. Agents can loop forever if you don't cap it.
- **RAG returns irrelevant chunks** — Use HyDE or query expansion - rewrite the user query first, then search. Or use top_k=5 with a relevance threshold.
- **Co-pay math is wrong** — Co-pay is applied to the COVERED amount, not the bill. covered = min(bill, sub_limit); payable = covered * (1 - co_pay).
- **User session lost on refresh** — Use st.session_state to persist chat history and uploaded PDF.

DEFINITION OF DONE

You can call this project complete when:

- [] User can upload a PDF and ask 5+ different question types successfully
- [] Agent correctly chains 2+ tools when needed (e.g., coverage check + claim calculation)
- [] Sample claim calculation matches the policy's actual sub-limit and co-pay rules
- [] Hospital lookup returns only cashless network hospitals
- [] Agent's reasoning is visible (verbose mode or 'show reasoning' toggle)
- [] App runs end-to-end on a fresh machine via README instructions

PLACEMENT POWER-UP

GAME-WINNING RESUME LINE

RESUME BULLET (steal this format)

"Built a multi-tool conversational AI agent for health insurance Q and A using RAG over policy documents combined with 6 custom tools - autonomously checks coverage, calculates claim amounts, and verifies hospital network status. Deployed via Streamlit with a Groq + LangChain backend."

Companies that hire for this

Insurers (largest hiring pool): HDFC Ergo, Star Health, Niva Bupa, ICICI Lombard, Acko, Tata AIG, Bajaj Allianz. **Insurtech:** Plum, PolicyBazaar, Coverfox, Onsurity, Vital. **FinTech adjacent:** Razorpay (insurance vertical), Cred, Paytm Insurance, Jio Insurance. **Healthcare AI:** Innovaccer, Practo, 1mg, PharmEasy. **Consulting practices working with these:** Deloitte, EY, Accenture, TCS BFSI, Infosys.

Interview questions to expect

Recruiters and engineers will ask about your project. Prep answers for these:

- Walk me through your agent loop. What happens when the user asks a question?
- How does your agent decide which tool to call first?
- What did you do to make sure the LLM doesn't hallucinate coverage details?
- How do you handle a question that needs information from multiple tools?
- Why did you choose ChromaDB over Pinecone or FAISS?
- If the policy PDF has 100 pages, how does your RAG keep performance reasonable?
- How would you extend this to support multiple policies per user?
- What's your evaluation strategy - how do you know the agent is giving correct answers?

BUILD IT. SHIP IT. WIN IT.

Push to GitHub. Add the resume bullet. Walk into your interview with a real Agentic AI project to talk about.